Data Structures Lab: Stacks and Queues

Instructions

- This lab focuses on implementation and understanding of **Stacks and Queues**.
- You are expected to **come prepared with the logic** of the problems.
- You may bring:
  - Class notes
  - Textbook material
  - Handwritten pseudocode or approach
- You are **NOT allowed** to bring:
  - Fully written code solutions
  - Code generated from LLMs or online platforms
- You may use:
  - Global variables
  - Function parameters
  - Basic pointers (for linked list implementations)
- This is a **hands-on + viva-based lab**.
  - You will be asked to **explain your logic** during the lab.
  - Marks will be based on **understanding, correctness, and clarity**.
- Focus on **making the logic work**, not writing perfect code.

Easy Level

1. Implement a stack using an array with:
   - push()
   - pop()
   - display()

2. Implement a queue using an array with:
    - enqueue()
    - dequeue()
    - display()
3. Extend your implementation to handle:
    - Overflow condition
    - Underflow condition
4. Reverse the elements of a stack using another stack.
5. Implement a function to peek the top element of a stack without removing it.
6. Count the number of elements in a stack or queue.

Medium Level

7. Implement a circular queue using an array.
8. Implement a stack using a linked list.
9. Implement a queue using a linked list.
10. Convert an infix expression to postfix expression.
11. Evaluate a postfix expression (support +, -, *, /).
12. Implement a queue using two stacks.
13. Implement a stack using two queues.
14. Sort a stack such that the top of the stack has the greatest element.

Hard Level (Attempt if comfortable)

15. Implement a double ended queue (deque) using an array.
16. Implement a priority queue using an array:
    - Either simple insertion with costly deletion
    - Or sorted insertion with simple deletion
17. Reverse the first k elements of a queue using a stack.
18. Design a **Special Stack** that supports:

- push()
- pop()
- isEmpty()
- isFull()
- getMin() — returns the minimum element in O(1) time

Constraint: Use O(1) extra space.

Example:

Stack: 16, 15, 29, 19, 18 (Top = 16)
getMin() → 15

After two pop operations:

Stack: 29, 19, 18 (Top = 29)
getMin() → 18

Viva Focus Areas

Be prepared to answer:

- Why did you choose this approach?
- What is the time complexity of your operations?
- What are the edge cases (overflow, underflow)?
- How would you improve your solution?

Note

> "In this lab, your ability to **explain** your solution matters more than writing perfect code."